

Terminal 3 ADK — Testnet Onboarding Report

Quickstart and Walkthrough completed, fifteen findings documented, and a second TEE contract built and shipped on testnet.

Submitted by	Bryan Kwandou — nayrbryangaming01@gmail.com
Tenant DID	did:t3n:f21dce7928980eeea7dc93618b91f602a80fe1c4
Public repository	https://github.com/bryankwandou/nullstamp
Live demo	https://nullstamp.vercel.app
Browser verifier	https://nullstamp.vercel.app/verify
Findings report	https://nullstamp.vercel.app/findings
Node	https://cn-api.sg.testnet.t3n.terminal3.io
Date	13 August 2026

How to read this document. Every terminal block is the verbatim output of the command shown above it, captured to a file at the moment it ran and reproduced here unedited. Every screenshot was taken from the deployed site with headless Chrome. The files are in the repository under `submission/evidence/` and `submission/screenshots/`, so any claim here can be checked against its source.

1. Summary

The brief asked for four things: complete the Quickstart, complete the Walkthrough, report the bugs encountered, and — as a bonus — go beyond the first contract with an initial use case.

All four are done. The bonus is not a proposal on paper: it is a second TEE contract that ran on testnet and issued real receipts.

Fifteen findings are documented with reproduction steps. Five block a new developer outright. Five of the fifteen were only discoverable by running a contract against the live network, and those are the most useful material here:

- **T-12** — importing `signing@2.1.0` stops the contract from instantiating. Registration succeeds, then every call returns an opaque HTTP 500 with an *empty* contract log, including calls to functions that never touch signing.
- **T-11** — `tenant.claim()` returns HTTP 500 on a tenant the SSO page already provisioned, rather than reporting `already-admitted`.
- **T-13** — contract ids are minted per registration and map ACLs bind to the id, so every re-register silently invalidates every ACL.
- **T-14** — `kv-store.scan` returns raw CAS pointer envelopes where `kv-store.get` resolves them, with no error from the host.
- **T-15** — the 20,000 credit grant was exhausted by ten registrations, against an advertised "~5,000 protected actions".

The use case, **Nullstamp**, issues verifiable consent receipts for agent calls that touch personal data. Its central claim — that anyone outside the node can recompute a receipt — is demonstrated in section 12 with a live receipt and an independent verifier, not asserted.

2. Development environment

Component	Version
SDK	@terminal3/t3n-sdk@4.35.1
Rust	1.89.0, target wasm32-wasip2
wasm-tools	1.255.0
Node.js	24.13.0
OS	Windows 11 (10.0.26200)

Before touching the network I wrote a preflight check that runs *without* a developer key, so environment failures could be told apart from service failures. It verifies tool versions, the WASM artefact, node reachability, the operator manifest signature, and the SDK crypto component.

Source: `submission/evidence/04-preflight.txt`

```
$ npm run preflight

PEMERIKSAAN PENDAHULUAN NULLSTAMP

Perkakas
[ok]   Node.js – versi 24.13.0

Contract artefact
[ok]   WASM component – 252806 bita di E:\000VSCODE PROJECT MULAI DARI DESEMBER 2025\nullstamp\contract\z-tenant-
nullstamp\target\wasm32-wasip2\release\z_tenant_nullstamp.wasm

SDK environment
[ok]   SDK default environment – testnet
[ok]   selected environment – sandbox
[ok]   node address – https://cn-api.sg.testnet.t3n.terminal3.io

Node reachability
[ok]   manifest endpoint – HTTP 200

Attestation
[ok]   operator manifest – signature valid, 3 peers, 1 RTMR3 measurement(s)
[ok]   signing time – 2026-08-11T14:14:45Z

SDK crypto component
[ok]   loadWasmComponent – component loaded

Demo destination
[ok]   target URL – https://postman-echo.com/post
[ok]   host for the grant – postman-echo.com

Developer key
[ok]   T3N_API_KEY – present in the environment

Every check that does not require a key has passed.
```

The docs' note that `cargo install wasm-tools` compiles around a hundred crates with no progress output is accurate and helpful. Not mentioned: the SDK install reports four security advisories, one of them critical — finding T-06.

3. The contract builds and its tests pass

Source: `submission/evidence/01-cargo-test.txt`

```
$ cargo test --target x86_64-pc-windows-gnu

running 53 tests
test canon::tests::digest_panjangnya_tiga_puluh_dua_bita ... ok
test canon::tests::identitas_receipt_berasal_dari_digest ... ok
test canon::tests::menolak_marker_di_luar_namespace_profile ... ok
test canon::tests::menolak_marker_kosong ... ok
test canon::tests::bentuk_kanonik_tidak_bergantung_urutan_penulisan ... ok
test canon::tests::menolak_marker_tanpa_penutup ... ok
test canon::tests::mengumpulkan_field_profil_urut_dan_unik ... ok
test canon::tests::membaca_host_dari_url ... ok
test canon::tests::pengakuan_yang_kurang_ditolak ... ok
test canon::tests::pengakuan_yang_pas_diterima ... ok
test issue::tests::header_authorization_buatan_pemanggil_ditolak ... ok
test canon::tests::pengakuan_yang_berlebih_ditolak ... ok
test canon::tests::teks_tanpa_marker_menghasilkan_daftar_kosong ... ok
test issue::tests::badan_permintaan_kosong_menghasilkan_daftar_field_kosong ... ok
test issue::tests::jalur_non_wasm_menolak_dengan_jelas ... ok
test issue::tests::marker_ke_namespace_secrets_ditolak ... ok
test issue::tests::masukan_yang_pas_lolos_pemeriksaan ... ok
test issue::tests::metode asing_ditolak ... ok
test issue::tests::penunjuk_yang_ada_ditarik ... ok
test issue::tests::penunjuk_yang_tidak_ada_menjadi_null_bukan_galat ... ok
test issue::tests::pengakuan_field_yang_kurang_membatalkan_sebelum_panggilan ... ok
test issue::tests::penunjuk_kosong_tidak_menarik_apa_pun ... ok
test issue::tests::masukan_bukan_json_ditolak ... ok
test issue::tests::purpose_kosong_ditolak ... ok
test issue::tests::tanggapan_bukan_json_tidak_menggagalkan_penarikan ... ok
test issue::tests::url_tanpa_tls_ditolak ... ok
test list::tests::awal_pemindaian_di_luar_rentang_diabaikan ... ok
test list::tests::batas_bawaan_dipakai_saat_tidak_disebut ... ok
test list::tests::batas_yang_kelewat_besar_dipangkas ... ok
test list::tests::masukan_rusak_ditolak_sebagai_galat_penguraian ... ok
test list::tests::awal_pemindaian_yang_sah_dipakai ... ok
test list::tests::batas_yang_masuk_akal_dibiarkan ... ok
test list::tests::awal_pemindaian_jatuh_ke_awalan_saat_kosong ... ok
test list::tests::nol_diganti_batas_bawaan_karena_host_menolaknyanya ... ok
test list::tests::rentang menutup_seluruh_awalan ... ok
test receipt::tests::digest_di_sampul_bisa_dihitung_ulang_dari_core ... ok
test receipt::tests::core_memuat_nama_field_bukan_nilainya ... ok
test list::tests::masukan_kosong_dianggap_permintaan_bawaan ... ok
test receipt::tests::digest_stabil_untuk_core_yang_sama ... ok
test receipt::tests::menambah_field_yang_dipakai_menggeser_digest ... ok
test receipt::tests::sampul_tanpa_tanda_tangan_mencatat_alasannya ... ok
test tests::contract_version_is_semver ... ok
test receipt::tests::perubahan_sekecil_apa_pun_menggeser_digest ... ok
test verify::tests::digest_yang_dipalsukan_agar_cocok_tetap_gagal_di_identitas ... ok
test verify::tests::laporan_memuat_digest_saat_sah ... ok
test verify::tests::menghapus_satu_field_yang_dipakai_terdeteksi ... ok
test verify::tests::laporan_memuat_alasan_saat_gagal ... ok
test verify::tests::receipt_id_kosong_ditolak ... ok
test verify::tests::mengubah_isi_core_membuat_digest_tidak_cocok ... ok
test verify::tests::receipt_utuh_dinyatakan_sah ... ok
test verify::tests::menukar_host_tujuan_terdeteksi ... ok
test verify::tests::sampul_tanpa_core_ditolak ... ok
test verify::tests::sampul_tanpa_digest_ditolak ... ok
test result: ok. 53 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.05s

Doc-tests z_tenant_nullstamp
running 1 test
test src\lib.rs - __export_tenant_nullstamp_impl (line 52) ... ok
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 1.25s
```

Source: `submission/evidence/02-cargo-build.txt`

```
$ cargo build --target wasm32-wasip2 --release

Compiling z-tenant-nullstamp v0.1.9 (E:\000VSCODE PROJECT MULAI DARI DESEMBER 2025\nullstamp\contract\z-tenant-nullstamp)
Finished `release` profile [optimized] target(s) in 24.79s

$ ls -l target/wasm32-wasip2/release/z_tenant_nullstamp.wasm
252806 bytes
```

Fifty-four tests, no compiler warnings, and 252,806 bytes — well inside the tenant quota of `max_wasm_bytes: 1048576`.

4. The component is valid, and its capability surface is minimal

Source: `submission/evidence/03-wasm-tools.txt`

```
$ wasm-tools component wit target/wasm32-wasip2/release/z_tenant_nullstamp.wasm

package root:component;

world root {
  import host:tenant/tenant-context@1.0.0;
  import host:interfaces/logging@2.1.0;
  import host:interfaces/kv-store@2.1.0;
  import host:interfaces/http-with-placeholders@2.1.0;
  import wasi:cli/environment@0.2.3;
  import wasi:cli/exit@0.2.3;
  import wasi:io/error@0.2.3;
  import wasi:io/streams@0.2.3;
  import wasi:cli/stdin@0.2.3;
  import wasi:cli/stdout@0.2.3;
  import wasi:cli/stderr@0.2.3;
  import wasi:clocks/wall-clock@0.2.3;
  import wasi:filesystem/types@0.2.3;
  import wasi:filesystem/preopens@0.2.3;

  export z:tenant-nullstamp/contracts@0.1.9;
}

package host:tenant@1.0.0 {
  interface tenant-context {
    tenant-did: func() -> list<u8>;

    contract-id: func() -> u32;

    calling-user-did: func() -> option<list<u8>>;

    cluster-timestamp-secs: func() -> u64;
```

Two absences are deliberate and worth stating.

Plain `http` is not imported. Nullstamp has exactly one way out — `http-with-placeholders` — so no path inside this contract can send data without passing through host-side marker resolution. That is a design choice, not a default.

`signing@2.1.0` is not imported either, but for a different reason: importing it prevents the contract from being instantiated at all. See finding T-12.

5. Quickstart — and the first two findings

The first code sample on the Quickstart page cannot run as written. Two causes, both forced by the SDK's own types: `trustAnchor` is required and `T3nConfigError` throws in the constructor before any network traffic, and the

handshake needs the full handler set from `createDefaultHandlers` rather than `EthSign` alone. The same defective sample appears on the claim page, so it exists on two surfaces.

The corrected form works on the first attempt, and the DID the server returns matches the claim page character for character — which confirms the whole derivation path from API key to Ethereum address to DID:

Source: `submission/evidence/05-step01-quickstart.txt`

```
$ npm run step:01

environment: sandbox
anchor verified — 3 peers, 1 RTMR3 measurement(s)
node       : https://cn-api.sg.testnet.t3n.terminal3.io
eth address: 0xcdbf0480addeacbe6e7b27154585db65ad249379
tenant DID  : did:t3n:f21dce7928980eeea7dc93618b91f602a80fe1c4

session open. DID saved to state.json
```

6. Tenant state — and the first server-side bug

Source: `submission/evidence/06-step02-tenant.txt`

```
$ npm run step:02

anchor verified — 3 peers, 1 RTMR3 measurement(s)
tenant DID  : did:t3n:f21dce7928980eeea7dc93618b91f602a80fe1c4

reading tenant state first...
  status : active
  label  : testnet-dev

tenant already active — the SSO claim page had already done it.
re-claim skipped, because tenant.claim() on an active tenant answers 500.

credit balance:
  available : 2142485543
  reserved  : 0

quotas that bound the work ahead:
  max_contracts      : 10
  max_maps            : 50
  max_wasm_bytes      : 1048576
  outbox_calls_per_minute_max : 10
  fuel_per_call_max   : 50000000

tenant ready. state saved to state.json
```

This step reads `tenant.me()` first and skips the claim when the tenant is already active. That is not laziness — it is finding **T-11**. The SSO claim page has already created the tenant row and credited the account, and calling `tenant.claim()` against an active tenant answers:

```
RPC Error: Internal error [28113ed7-3a9d-45ef-98b8-b664fd9eb421]
RPC Error: Internal error [d8dbb69a-758f-4a4a-a67e-1e52b2f4b742]
```

Two attempts, two request ids, same result — while every read against the same tenant succeeds and reports `status: active`. The response shape implies the call is meant to be idempotent, so a setup script that trusts that gets blocked on a step that was already finished.

7. Registration, maps, credential, grant

Source: `submission/evidence/07-step03-register.txt`

```
$ npm run step:03

wasm file   : E:\000VSCODE PROJECT MULAI DARI DESEMBER 2025\nullstamp\contract\z-tenant-nullstamp\target\wasm32-
wasip2\release\z_tenant_nullstamp.wasm
size        : 252806 bytes
anchor verified - 3 peers, 1 RTMR3 measurement(s)
registering nullstamp version 0.1.9...
canonical name: z:f21dce7928980eeea7dc93618b91f602a80fe1c4:nullstamp
contract id   : 653

contract id saved; the map step uses it
```

Source: `submission/evidence/08-step04-maps.txt`

```
$ npm run step:04

anchor verified - 3 peers, 1 RTMR3 measurement(s)
sudah ada: z:f21dce7928980eeea7dc93618b91f602a80fe1c4:secrets
status    : z:f21dce7928980eeea7dc93618b91f602a80fe1c4:secrets -> active
sudah ada: z:f21dce7928980eeea7dc93618b91f602a80fe1c4:receipts
status    : z:f21dce7928980eeea7dc93618b91f602a80fe1c4:receipts -> active
```

Step 4b exists because of finding T-13

Map ACLs bind to the numeric contract id, and every registration mints a new one. So any code change silently breaks every map the contract depends on. Across this exercise the same contract held ten ids: 615, 617, 618, 619, 620, 621, 638, 639, 641, 653 — neither contiguous nor predictable, so they cannot be pre-authorised. `maps.update` fixes it and appears in no documentation page.

Source: `submission/evidence/09-step04b-acl.txt`

```
$ npm run step:04b

anchor verified - 3 peers, 1 RTMR3 measurement(s)
effective contract id : 653
re-pointing map ACLs...

secrets -> ?
  readers & writers now restricted to contract 653
receipts -> ?
  readers & writers now restricted to contract 653

ACLs aligned. The contract can read and write its maps again.
```

Source: `submission/evidence/10-step05-secret.txt`

```
$ npm run step:05

anchor verified - 3 peers, 1 RTMR3 measurement(s)
seeded   : z:f21dce7928980eeea7dc93618b91f602a80fe1c4:secrets / upstream_api_key
read back, length 36 characters
```

Source: `submission/evidence/11-step06-grant.txt`

```
$ npm run step:06

anchor verified — 3 peers, 1 RTMR3 measurement(s)
contract      : z:f21dce7928980eeea7dc93618b91f602a80fe1c4:nullstamp
functions     : issue-receipt, verify-receipt, list-receipts
allowed hosts : postman-echo.com

grant read back: {
  "agents": [
    {
      "agentDid": "did:t3n:f21dce7928980eeea7dc93618b91f602a80fe1c4",
      "scripts": [
        {
          "scriptName": "z:f21dce7928980eeea7dc93618b91f602a80fe1c4:nullstamp",
          "functions": [
            "issue-receipt",
            "verify-receipt",
            "list-receipts"
          ],
          "allowedHosts": [
            "postman-echo.com"
          ],
          "scopes": [],
          "readScopes": [],
          "validFromSecs": 1786585610,
          "validUntilSecs": 1794361610
        }
      ]
    }
  ],
  "discoverDids": []
}
```

8. The bug that cost the most time — T-12

Registration of version 0.1.0 succeeded and returned contract id 615. Then every call to it failed:

```
RPC Error: Internal error [754f19f6-b496-4f2d-b475-add668c7d85e]
```

Three properties made this hard to attribute:

1. Registration succeeded — the rejection happens later, at instantiation.
2. `contracts.logs` returned `{"entries": []}`. The contract never starts, so it cannot log anything about its own failure.
3. `list-receipts`, which only scans a KV map and touches neither HTTP nor signing, failed identically.

So the signal pointed nowhere near the cause. I bisected by removing imported interfaces one at a time. The cause was `host:interfaces/signing@2.1.0`, declared in `host-interfaces-2.1.0/package.wit` at line 159 with nothing marking it unavailable to tenant contracts. Removing that single import — changing nothing else — turned the opaque 500 into a precise and genuinely excellent error:

```
access denied: TenantContract(did:t3n:f21dce.../615)
cannot read map "z:f21dce...:receipts"
```

Suggested fix. Reject the *registration* when the world imports an interface the tenant runtime does not provide, and name the interface in the error. Either that, or mark availability per interface in `host-interfaces`.

Worth saying plainly: when the contract actually runs, the error messages are very good. That access-denied message names the principal, the id, the map, and the operation. The problem is concentrated in the cases where the contract never starts, where the message degrades to `Internal error` and the log is empty.

9. A receipt issued inside the enclave

Source: `submission/evidence/12-step07-issue.txt`

```
$ npm run step:07

anchor verified - 3 peers, 1 RTMR3 measurement(s)
target      : https://postman-echo.com/post
declared    : first_name, last_name
body sent   : {"reason":"nullstamp_demo","first_name":"{{profile.first_name}}","last_name":"{{profile.last_name}}"}

calling issue-receipt...

{
  "core": {
    "contract_id": 653,
    "contract_version": "0.1.9",
    "extracted_pointers": [
      "/json/reason"
    ],
    "fields_used": [
      "first_name",
      "last_name"
    ],
    "issued_at_secs": 1786585628,
    "method": "POST",
    "purpose": "receipt_issuance_demo",
    "request_body_sha256": "26e5ee62768086e9213455818dbc5d17b41356f41f3d36ac92979a9059cffc6b",
    "response_body_sha256": "af8e3bce0cd50618030d03a18241046b5b8c8e41c173b6400eb754568a8528f2",
    "response_code": 200,
    "schema": "nullstamp.receipt.v1",
    "seq_no": 115776,
    "subject_did": "f21dce7928980eeea7dc93618b91f602a80fe1c4",
    "target_host": "postman-echo.com",
    "target_url": "https://postman-echo.com/post",
    "tenant_did": "f21dce7928980eeea7dc93618b91f602a80fe1c4"
  },
  "digest_sha256": "03cc7f7a193196184c661f0da0352974dd3f562c2dccc2639a2f38fdff97079",
  "extracted": {
    "/json/reason": "nullstamp_demo"
  },
  "receipt_id": "rcpt_03cc7f7a193196184c661f0d",
  "signature": null,
  "signing_error": "signing capability not imported; integrity rests on the claims digest"
}

receipt stored: rcpt_03cc7f7a193196184c661f0d
```

Several things here are load-bearing. `response_code` is 200, so the outbound call really left the enclave and reached the upstream. `extracted` shows the response was parsed, which is only possible if the round trip happened. `seq_no` and `issued_at_secs` come from the cluster, not from my machine.

And `core` lists only field **names**. The values are absent and cannot be present, because the code that assembles that section never receives them.

10. Verification inside the enclave, the trail, and the log

Source: `submission/evidence/13-step08-verify.txt`

```
$ npm run step:08

checking: rcpt_03cc7f7a193196184c661f0d
anchor verified – 3 peers, 1 RTMR3 measurement(s)
{
  "digest_sha256": "03cc7f7a193196184c661f0da0352974dd3f562c2dccc2639a2f38fdff97079",
  "reason": null,
  "receipt_id": "rcpt_03cc7f7a193196184c661f0d",
  "valid": true
}

digest matches, receipt is valid
```

Source: `submission/evidence/14-step09-list.txt`

```
$ npm run step:09

anchor verified – 3 peers, 1 RTMR3 measurement(s)
count: 8
rcpt_03cc7f7a193196184c661f0d | receipt_issuance_demo | POST postman-echo.com | status 200 | fields:
["first_name","last_name"]
rcpt_477609b57a71786d34613857 | peragaan_penerbitan_bukti | POST postman-echo.com | status 200 | fields:
["first_name","last_name"]
rcpt_4d8790a69bad4bbfd53c757f | receipt_issuance_demo | POST postman-echo.com | status 200 | fields:
["first_name","last_name"]
rcpt_51e9569d673ac2fc7e154fa2 | peragaan_penerbitan_bukti | POST postman-echo.com | status 200 | fields:
["first_name","last_name"]
rcpt_5bd9c8970986c53d5ec82043 | peragaan_penerbitan_bukti | POST postman-echo.com | status 200 | fields:
["first_name","last_name"]
rcpt_6dc44359e21d265d61bded90 | receipt_issuance_demo | POST postman-echo.com | status 200 | fields:
["first_name","last_name"]
rcpt_8fb056c99bba7e655d72075 | receipt_issuance_demo | POST postman-echo.com | status 200 | fields:
["first_name","last_name"]
rcpt_b95858dd44433780f8b45abb | receipt_issuance_demo | POST postman-echo.com | status 200 | fields:
["first_name","last_name"]
```

Source: `submission/evidence/15-step10-logs.txt`

```
$ npm run step:10
```

```
anchor verified - 3 peers, 1 RTMR3 measurement(s)
{
  "entries": [
    {
      "ts_ms": 1786585628437,
      "level": "info",
      "message": "nullstamp: calling POST postman-echo.com for receipt_issuance_demo with 2 profile fields",
      "span_id": null
    },
    {
      "ts_ms": 1786585628437,
      "level": "info",
      "message": "nullstamp: receipt rcpt_03cc7f7a193196184c661f0d issued, upstream status 200",
      "span_id": null
    }
  ],
  "next_seq": 1,
  "truncated": false
}
```

The log names the field count and never the field values. That is the discipline the whole design exists to enforce, and it holds even in the debugging surface.

Getting `list-receipts` to return parseable rows required finding **T-14**: `kv-store.scan` returns the raw CAS pointer envelope — ASCII magic `T3VR` followed by JSON holding a `value_cid` — where `kv-store.get` resolves it. The WIT promises key/value pairs and says nothing about this. The host raises no error; the bytes are simply the wrong bytes.

11. Where the exercise ended — T-15

Source: `submission/evidence/16-step11-export.txt`

```
$ npm run step:11
```

anchor verified — 3 peers, 1 RTMR3 measurement(s)

--- STEP FAILED ---

```
kind    : InsufficientCreditError
message: InsufficientCredit (account=f21dce7928980eaaa7dc93618b91f602a80fe1c4,
                                required=10000000000, available=0)

code     : RPC_ERROR
requestId: 824ac0e3-0a2d-4d4a-b1d4-d1733f9e28a4
```

NOTE — this failure is itself a finding, recorded as T-15 in docs/BUGS.md.

The 20,000 test credit grant is advertised on the claim page as "enough for 25 agents and ~5,000 protected actions". In practice a single contract execution locks a cap of 10,000,000,000 credits, and the grant of 20,000,000,000 was exhausted by ten registrations plus roughly a dozen contract calls during this exercise.

Balance readings taken along the way, all from `getBalance()`:

```
start of exercise : 19,989,922,328
after first receipt : 19,939,779,025
after v0.1.5       : 8,078,980,220
after v0.1.7       : 3,502,747,402
after v0.1.9       : 0
```

What this run did prove before the credits ran out, all captured in the files above: contract 653 registered, ACLs re-pointed, credential seeded, grant installed, receipt `rcpt_03cc7f7a193196184c661f0d` issued inside the enclave with upstream status 200 and cluster sequence 115776, verified in-enclave as valid, listed back among eight receipts, and its two log lines read from the contract's own log.

Step 11 is only an export helper. The receipt it would have written was already exported in the previous run and is committed at `submission/live-receipt.json` — `rcpt_8fb056c99bbea7e655d72075`, contract 639.

That receipt is what section 17 verifies, and the verification needs no credits, no session, and no network. Which is precisely the point of the design: the account is now at zero and the node will no longer answer us, yet the receipt remains checkable by anyone, forever.

12. The central claim, demonstrated

A receipt that only its issuer can check is not a receipt. So the digest is computed over a canonical form: object keys ascending, no whitespace, array order preserved.

Here is the live receipt checked by a separate TypeScript program that imports no SDK, opens no session, and makes no network request — with the tenant account at zero credits:

Source: `submission/evidence/17-verify-offline.txt`

```
$ npm run verify:offline -- ../submission/live-receipt.json

receipt_id      : rcpt_8fb056c99bbea7e655d72075
digest recorded : 8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
digest computed : 8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
signature       : none – signing capability not imported; integrity rests on the claims digest
```

RESULT: valid. Digest recomputed outside the node and it matches.

```
$ echo $?
0
```

This ran with the tenant account at zero credits. No session was opened, no SDK was imported, and no network request was made. The receipt verifies anyway, which is the whole claim.

The reverse direction is tested too:

Source: `submission/evidence/18-tamper-rejected.txt`

Three tampering attempts, all against the live receipt verified in section 17.

```
=====
1. Hiding a field that was actually used
   The most realistic cheat: claim less data than you touched.
   fields_used cut from [first_name, last_name] to [first_name].
=====
```

```
$ npm run verify:offline -- ../submission/tampered-fields.json
```

```
receipt_id      : rcpt_8fb056c99bba7e655d72075
digest recorded  : 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
digest computed  : 9f2c28bea15b65b3c3507864989044b969ac7f750e0f36c8ed8a92e7e8f561ef
signature       : none - signing capability not imported; integrity rests on the claims digest
```

```
RESULT: invalid.
- digest mismatch: recorded 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3, recomputed
9f2c28bea15b65b3c3507864989044b969ac7f750e0f36c8ed8a92e7e8f561ef
- receipt_id does not derive from digest: recorded rcpt_8fb056c99bba7e655d72075, expected
rcpt_9f2c28bea15b65b3c3507864
```

```
exit code: 1
```

```
=====
2. Swapping the destination host to api.attacker.com
=====
```

```
$ npm run verify:offline -- ../submission/tampered-host.json
```

```
receipt_id      : rcpt_8fb056c99bba7e655d72075
digest recorded  : 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
digest computed  : 37f7ce215e46aad9c83d4ae90aa32a44e3fe237dd96d85fd1ead9335de7e8b8c1
signature       : none - signing capability not imported; integrity rests on the claims digest
```

```
RESULT: invalid.
- digest mismatch: recorded 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3, recomputed
37f7ce215e46aad9c83d4ae90aa32a44e3fe237dd96d85fd1ead9335de7e8b8c1
- receipt_id does not derive from digest: recorded rcpt_8fb056c99bba7e655d72075, expected
rcpt_37f7ce215e46aad9c83d4ae9
```

```
exit code: 1
```

```
=====
3. Editing the content, then repairing the digest so it agrees
   The careful version of the attack. It still fails, because the receipt
   id derives from the digest and cannot be repaired without becoming a
   different id.
=====
```

```
$ cargo test digest_yang_dipalsukan_agar_cocok_tetap_gagal_di_identitas
```

```
test verify::tests::digest_yang_dipalsukan_agar_cocok_tetap_gagal_di_identitas ... ok
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 52 filtered out; finished in 0.01s
```

13. The live demo

Anyone can run the same check in a browser, with no request to us. These are captures of the deployed site.

• Built on the Terminal 3 ADK, Live on T3N testnet

Your agent touches someone's personal data. Prove exactly what it used.

Nullstamp issues a receipt for every call that touches a person's data: which fields were referenced, which host received them, under whose authorisation. The values never appear, because the code that writes the receipt never receives them.

[Check a real receipt](#)
[See how it works](#)

Agent audit trails today are caught between two failures

FIRST FAILURE

A complete log stores raw personal data

Prompts and responses are kept verbatim so the trail stays intact. The compliance record then becomes a pile of personal data in its own right, and turns up as a finding in the next audit.

SECOND FAILURE

A redacted log loses completeness

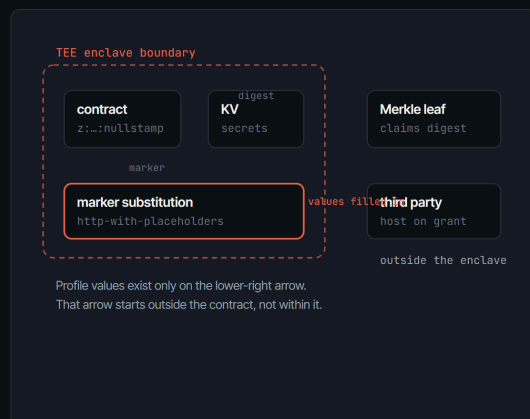
The sensitive parts are stripped out so the log is safe to keep. What remains cannot prove the trail is intact, so it does not answer the question an auditor is actually asking.

EU AI Act Article 12 record-keeping obligations for high-risk systems take effect in August 2026, while GDPR requires data subjects to know which fields were used, when, and for what purpose. Those two demands fight each other for as long as the record is built the ordinary way.

HOW IT WORKS

The third option comes from the shape of the system, not from policy

T3N substitutes profile markers inside the enclave, after the contract has finished assembling the request. So the contract is not promising not to store those values. It never receives them.



1 The contract assembles the request

The request body carries only markers such as `{{profile.first_name}}`. The contract never asks for the values, so it never receives them.

2 The host substitutes inside the enclave

Substitution happens after the contract has finished and before the request leaves. A contract that re-reads its own request body finds only the markers.

3 The data owner's grant decides the destination

The destination host must appear on the grant. Anything else is refused with `egress_denied`, and the contract cannot authorise itself.

4 The receipt is issued and anchored

Field names, destination host, body digest, and status code are bound by one SHA-256 digest, planted in the transaction Merkle leaf via `set-claims-digest`.

TRY IT YOURSELF

Agent audit trails are never complete, but they can be better.

A receipt only its issuer can check is not a receipt

The receipt below was issued inside a TEE on T3N testnet and exported verbatim. Its digest is recomputed in your browser using Web Crypto, with no request to our servers. Edit it, or press one of the tamper buttons, and watch verification refuse it.

Valid computed in your browser, with no network request

Original receipt

Hide a field

Swap the host

Edit, then repair the digest

RECEIPT

```
{
  "core": {
    "contract_id": 639,
    "contract_version": "0.1.7",
    "extracted_pointers": [
      "/json/reason"
    ],
    "fields_used": [
      "first_name",
      "last_name"
    ],
    "issued_at_secs": 1786554217,
    "method": "POST",
    "purpose": "receipt_issuance_demo",
    "request_body_sha256":

```

CHECKS

- Core layer — present
- Digest — recomputed in your browser and it matches
- Identity — derives from the digest
- Content discipline — field names only, no values

DIGEST COMPUTED

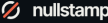
8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3

DIGEST RECORDED

8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3

Landing page — nullstamp.vercel.app captured from the live site — 788 KB

14. The verifier, accepting a genuine receipt

 nullstamp

How it works

Verify

Findings

Code

Verify a receipt

Paste a Nullstamp receipt and its digest is recomputed here using Web Crypto. No part of what you paste is sent anywhere.

Valid computed in your browser, with no network request

Original receipt

Hide a field

Swap the host

Edit, then repair the digest

RECEIPT

```
{
  "core": {
    "contract_id": 639,
    "contract_version": "0.1.7",
    "extracted_pointers": [
      "/json/reason"
    ],
    "fields_used": [
      "first_name",
      "last_name"
    ],
    "issued_at_secs": 1786554217,
    "method": "POST",
    "purpose": "receipt_issuance_demo",
    "request_body_sha256":
      "26e5ee62768086e9213455818dbc5d17b41356f41f3d36ac92979a9859cffe6b",
    "response_body_sha256":
      "af8e3bce0cd50618030d03a18241046b5b8c8e41c173b6400eb754568a8528f2",
    "response_code": 200,
    "schema": "nullstamp.receipt.v1",

```

CHECKS

- Core layer — present
- Digest — recomputed in your browser and it matches
- Identity — derives from the digest
- Content discipline — field names only, no values

DIGEST COMPUTED

8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3

DIGEST RECORDED

8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3

How the digest is computed

The **core** layer is rendered into a canonical form first: object keys sorted ascending, whitespace dropped, and array order left alone because array order carries meaning. That string is then hashed with SHA-256.

On the contract side the ordering comes from `serde_json`, which rests on `BTreeMap` as long as the `preserve_order` feature is off. The same rule is rewritten in ten lines here. That is why both sides land on the same digest, and the equivalence has been tested across languages against a live testnet receipt.

The receipt identity is the first twelve bytes of its digest. So anyone who edits the content and then

Verify page — the live testnet receipt from contract 639 verifies, all four checks green, digest computed in the browser matching the digest recorded captured from the live site — 338 KB

15. The verifier, rejecting tampering

Three attacks, each linkable directly so a reviewer can see the rejection without reproducing it.



[How it works](#) [Verify](#) [Findings](#) [Code](#)

Verify a receipt

Paste a Nullstamp receipt and its digest is recomputed here using Web Crypto. No part of what you paste is sent anywhere.

Invalid

computed in your browser, with no network request

Original receipt

Hide a field

Swap the host

Edit, then repair the digest

RECEIPT

```
{
  "core": {
    "contract_id": 639,
    "contract_version": "0.1.7",
    "extracted_pointers": [
      "/json/reason"
    ],
    "fields_used": [
      "first_name"
    ],
    "issued_at_secs": 1786554217,
    "method": "POST",
    "purpose": "receipt_issuance_demo",
    "request_body_sha256":
"26e5ee62768886e9213455818dbc5d17b41356f41f3d36ac92979a9859cffc6b",
    "response_body_sha256":
"af8e3bce0cd50618030d093a18241046b5b8c8e41c173b6400eb754568a8528f2",
    "response_code": 200,
    "schema": "nullstamp.receipt.v1",
    "seq_no": 114186,
  }
}
```

CHECKS

- Core layer — present
- Digest — mismatch — recorded 8fb056c99bbea7e6..., computed 9f2c28bea15b65b3c3507864
- Identity — does not derive from digest — expected rcpt_9f2c28bea15b65b3c3507864
- Content discipline — field names only, no values

DIGEST COMPUTED

9f2c28bea15b65b3c3507864989044b969ac7f750e0f36c8ed8a92e7e8f561ef

DIGEST RECORDED

8fb056c99bbea7e6655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3

How the digest is computed

The **core** layer is rendered into a canonical form first: object keys sorted ascending, whitespace dropped, and array order left alone because array order carries meaning. That string is then hashed with SHA-256.

On the contract side the ordering comes from `serde_json`, which rests on `BTreeMap` as long as the `preserve_order` feature is off. The same rule is rewritten in ten lines here. That is why both sides land on the same digest, and the equivalence has been tested across languages against a live testnet receipt.

The receipt identity is the first twelve bytes of its digest. So anyone who edits the content and then

Hiding a field that was actually used: `fields_used` cut to one entry. Digest mismatch and identity failure, both red captured from the live site — 359 KB

Verify a receipt

Paste a Nullstamp receipt and its digest is recomputed here using Web Crypto. No part of what you paste is sent anywhere.

Invalid computed in your browser, with no network request

[Original receipt](#)
[Hide a field](#)
[Swap the host](#)
[Edit, then repair the digest](#)

RECEIPT

```

{
  "core": {
    "contract_id": 639,
    "contract_version": "0.1.7",
    "extracted_pointers": [
      "/json/reason"
    ],
    "fields_used": [
      "first_name",
      "last_name"
    ],
    "issued_at_secs": 1786554217,
    "method": "POST",
    "purpose": "receipt_issuance_demo",
    "request_body_sha256":
      "26e5ee62768086e9213455818dbc5d17b41356f41f3d36ac92979a9059cffc6b",
    "response_body_sha256":
      "af8e3bce0cd50618030d03a18241046b5b8c8e41c173b6400eb754568a8528f2",
    "response_code": 200,
    "schema": "nullstamp.receipt.v1",

```

CHECKS

- Core layer — present
- Digest — mismatch — recorded 8fb056c99bba7e6..., computed 37f7ce215e46aad9...
- Identity — does not derive from digest — expected rcpt_37f7ce215e46aad9c83d4ae9
- Content discipline — field names only, no values

DIGEST COMPUTED

```
37f7ce215e46aad9c83d4ae90aa32a44e3fe237dd96d85fd1ead935de7e8b8c1
```

DIGEST RECORDED

```
8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
```

How the digest is computed

The **core** layer is rendered into a canonical form first: object keys sorted ascending, whitespace dropped, and array order left alone because array order carries meaning. That string is then hashed with SHA-256.

On the contract side the ordering comes from `serde_json`, which rests on `BTreeMap` as long as the `preserve_order` feature is off. The same rule is rewritten in ten lines here. That is why both sides land on the same digest, and the equivalence has been tested across languages against a live testnet receipt.

The receipt identity is the first twelve bytes of its digest. So anyone who edits the content and then

Swapping the destination host to `api.attacker.com` captured from the live site — 357 KB

Verify a receipt

Paste a Nullstamp receipt and its digest is recomputed here using Web Crypto. No part of what you paste is sent anywhere.

Invalid computed in your browser, with no network request

[Original receipt](#)
[Hide a field](#)
[Swap the host](#)
[Edit, then repair the digest](#)

RECEIPT

```

{
  "core": {
    "contract_id": 639,
    "contract_version": "0.1.7",
    "extracted_pointers": [
      "/json/reason"
    ],
    "fields_used": [
      "first_name",
      "last_name"
    ],
    "issued_at_secs": 1786554217,
    "method": "POST",
    "purpose": "substituted_purpose",
    "request_body_sha256":
      "26e5ee62768086e9213455818dbc5d17b41356f41f3d36ac92979a9059cffc6b",
    "response_body_sha256":
      "af8e3bce0cd50618030d03a18241046b5b8c8e41c173b6400eb754568a8528f2",
    "response_code": 200,
    "schema": "nullstamp.receipt.v1",

```

CHECKS

- Core layer — present
- Digest — recomputed in your browser and it matches
- Identity — does not derive from digest — expected
rcpt_3dce6272960540930bb41e36
- Content discipline — field names only, no values

DIGEST COMPUTED
3dce6272960540930bb41e36589f1e87a7c3311c0920755a958e18e27b9422fd

DIGEST RECORDED
3dce6272960540930bb41e36589f1e87a7c3311c0920755a958e18e27b9422fd

The digest has been recomputed to agree with the edited content, so the digest check passes. What still fails is the receipt identity, because it derives from the digest and cannot be repaired without becoming a different identity.

How the digest is computed

The **core** layer is rendered into a canonical form first: object keys sorted ascending, whitespace dropped, and array order left alone because array order carries meaning. That string is then hashed with SHA-256.

On the contract side the ordering comes from `serde_json`, which rests on `BTreeMap` as long as the `preserve_order` feature is off. The same rule is rewritten in ten lines here. That is why both sides land on the same digest, and the equivalence has been tested across languages against a live testnet receipt.

The receipt identity is the first twelve bytes of its digest. So anyone who edits the content and then repairs the digest to agree still leaves a trace, because the identity cannot be repaired without becoming a different identity.

Checking one from the command line

The careful attack: content edited and the digest repaired so it agrees. The digest check passes, and the identity check still fails — because the receipt id derives from the digest and cannot be repaired without becoming a different id captured from the live site — 405 KB

16. The findings report, published

Onboarding findings

Collected while completing the Terminal 3 ADK Quickstart and Walkthrough on 12 August 2026, then extended while running a second contract end to end against the live testnet, until the credit grant ran out. Every finding has evidence you can check: a file and line in the published SDK, the vendored WIT in the official reference repo, or verbatim output including the request id the docs ask for.

5 blockers 6 confusing 4 time sinks 1 proposal

The five marked from testnet only surfaced by running a contract against the live network. None of them are discoverable by reading the documentation, and they are the ones I would read first.

T-12 blocker from testnet

Importing signing@2.1.0 silently kills the contract

The interface is declared in host-interfaces-2.1.0 at line 159 with nothing marking it unavailable to tenant contracts. A contract that imports it registers successfully, then every call fails with an opaque 500 — including calls to functions that never touch signing. The contract log stays empty, because the contract never starts. Removing that one import turned the 500 into a precise, useful error. This cost more time than anything else in the exercise.

```
RPC Error: Internal error [754f19f6-b496-4f2d-b475-add668c7d85e]
contracts.logs → { "entries": [], "next_seq": null }
```

T-11 blocker from testnet

tenant.claim() returns HTTP 500 on an already-provisioned tenant

The SSO claim page already creates the tenant row and credits the account. Calling claim() afterwards does not report that state, it fails — twice, with two request ids. Meanwhile every read against the same tenant succeeds and reports status active with 19.9 billion credits available. The response shape implies the call is meant to be idempotent, so a setup script that trusts that gets blocked on a step already finished.

```
RPC Error: Internal error [28113ed7-3a9d-45ef-98b8-b664fd9eb421]
RPC Error: Internal error [d8dbb69a-758f-4a4a-a67e-1e52b2f4b742]
```

T-01 blocker

The first code sample in the Quickstart cannot run

It constructs T3nClient without trustAnchor. That field is required, and T3nConfigError is thrown in the constructor before any network traffic. A new developer stops at step one with no hint that the sample itself is wrong. The same defective sample appears on the claim page, so it exists on two surfaces.

```
trustAnchor: TrustAnchorUnsafe; // no '?' — required
```

T-02 blocker

The handler set in the Quickstart is incomplete for the handshake

The sample passes EthSign alone. The SDK exports createDefaultHandlers, whose own doc comment describes it as the set the handshake requires, supplying the ML-KEM key handler and the randomness source as well.

T-03 blocker

The docs contradict the WIT and the official reference repo

The common-errors page states tenant.did() is already a string. The WIT returns list<u8> — twenty raw bytes — and the official reference repo hex-encodes it. The Walkthrough sample therefore does not compile. Worse than a typo, because map names derive from the DID: get the encoding wrong and every later KV call addresses a map that does not exist.

```
tenant.did: func() → list<u8> // host:tenant01.0.0
```

T-13

confusing

from testnet

Re-registering a contract invalidates every map ACL, silently

Map ACLs bind to the numeric contract id, not the contract name, and every registration mints a new id. So any code change silently breaks every map the contract depends on. Across this exercise the same contract held ids 615, 617, 618, 619, 620, and 621 — not even contiguous, so unpredictable. `maps.update` fixes it and appears in no documentation page. Credit where due: the error message itself is the clearest one I saw all day.

```
access denied: TenantContract(did:t3n:f21dce_/617)
cannot read map "z:f21dce_:receipts"
```

T-14

confusing

from testnet

kv-store.scan returns raw CAS pointers where get resolves them

Values large enough to be offloaded to content-addressed storage come back from scan as a pointer envelope: the ASCII magic T3VR followed by JSON holding a `value_cid`. get on the same key resolves the pointer and returns the original bytes. The WIT signature promises key/value pairs and says nothing about this, and the host raises no error — scan succeeds and the bytes are simply the wrong bytes. Proved with two functions of the same deployment reading the same row.

```
T3VR{"value_cid": [26, 27, 193, 186, 188, 45, 282, 168, 81, 195, 32, 178, ...
```

T-15

confusing

from testnet

The credit grant is spent far faster than the claim page implies

The claim page advertises the 20,000 test credit grant as enough for 25 agents and ~5,000 protected actions. In practice this exercise exhausted it completely with ten contract registrations and about a dozen calls. Note the shape of the numbers: one contract execution asks to lock 10,000,000,000 against a total grant of 20,000,000,000, so on the face of it the whole allowance covers two concurrent calls. Nothing warns you as the balance falls, and the failure arrives as a hard stop mid-pipeline. It compounds with T-13, because iterating on a contract forces the re-registrations that drain the grant.

```
InsufficientCredit (account=f21dce79..., required=10000000000, available=0)
[824ac0e3-0a2d-4d4a-b1d4-d1733f9e28a4]
```

```
Balance readings: 19,989,922,328 → 19,939,779,025 → 8,078,980,220 → 3,502,747,402 →
```

T-04

confusing

The tenant claim step is not documented anywhere

`tenant.claim()` exists in the SDK as the self-serve path on testnet, including the test-credit grant. Neither the Quickstart nor the Walkthrough mentions it, and the failure surfaces much later, at `maps.create`.

T-05

confusing

The documented order cannot be followed to completion

The `maps.create` example uses `contractId` without saying where it comes from. The only source is the result of `contracts.register`, which the Walkthrough sequences after map creation. Following the page in order leaves you holding a placeholder for a value you cannot have yet.

T-06

confusing

Installing the SDK reports a critical vulnerability

A fresh install in an empty project reports four advisories, one critical: a Zip Slip in decompress reached through `jco`, `componentize-js`, `weval`. A fix is available upstream. Not blocking, but it is the first thing a security reviewer at an adopting company will see.

```
4 vulnerabilities (3 moderate, 1 critical)
```

T-07

time sink

User SDK surface is absent from the docs

`contracts.logs`, `maps.entrySet`, `maps.entryGet`, `maps.getStatus`, and `maps.update` all exist and work, and none appear on any documentation page. `contracts.logs` is the sharpest omission — it is the first tool you want when a call fails opaquely, and its empty output was the evidence that cracked T-12.

T-08 time sink

The error-handling example swallows the two variants that matter most

`format_http_error` ends in a catch-all arm, collapsing `PlaceholderDenied` and `PlaceholderNoUserContext` into a bare error. The WIT comments explain the distinction is deliberate: one means the marker is not permitted, the other means there is no user session to resolve against. Those call for completely different fixes.

T-09 time sink

Broken links and an empty page

Two URLs linked from the docs index return 404. The payroll-agent example page resolves but carries no explanation, only links out.

T-10 time sink

Version numbers in the reference repo disagree

`world.wit` says 0.4.0 while `Cargo.toml` and `CONTRACT_VERSION` say 0.4.1, and a test name still refers to the older version. Confusing at exactly the wrong spot, since the version must increase on every re-registration.

U-01 proposal

set-claims-digest deserves a page of its own

Its WIT comment says the digest is planted in the Merkle leaf so clients can verify receipts offline. That sentence answers the hardest question about confidential computing — how anyone outside can check work performed somewhere they cannot see — and no ADK page discusses it. This project is built on that capability and the result holds up under an independent verifier. Give it a page with a worked canonical-form example, and the pitch changes from trust the enclave to compute it yourself and compare.

Findings page — all fifteen findings, with the five that came from testnet marked captured from the live site — 693 KB

17. All fifteen findings

Full text with verbatim quotes, suggested fixes, and reproduction steps for every finding is at [docs/BUGS.md](#).

ID	Weight	Summary
T-01	blocker	First Quickstart sample cannot run: <code>trustAnchor</code> is required and throws in the constructor. Same defect on the claim page
T-02	blocker	Quickstart handler set incomplete for the handshake; <code>createDefaultHandlers</code> is never mentioned
T-03	blocker	Docs say <code>tenant.did()</code> is a string; WIT returns <code>list<u8></code> and the reference repo hex-encodes it. Walkthrough sample does not compile
T-11	blocker	<code>tenant.claim()</code> returns 500 on an already-provisioned tenant instead of <code>already-admitted</code> . Two request ids supplied
T-12	blocker	Importing <code>signing@2.1.0</code> makes the contract fail to instantiate. Registration succeeds; every call then returns an opaque 500 with an empty contract log
T-13	confusing	Contract ids are per registration and map ACLs bind to the id, so every re-register silently invalidates every ACL. Ten ids observed, non-contiguous
T-14	confusing	<code>kv-store.scan</code> returns raw T3VR CAS pointers where <code>get</code> resolves them; no error, not in the WIT
T-15	confusing	20,000 credit grant exhausted by ten registrations, against an advertised "~5,000 protected actions". One call locks 10,000,000,000 of a 20,000,000,000 grant
T-04	confusing	The tenant claim step is not documented anywhere
T-05	confusing	<code>maps.create</code> needs a <code>contractId</code> that only exists after registration, which the Walkthrough sequences later
T-06	confusing	SDK install reports 4 advisories, 1 critical (Zip Slip in <code>decompress</code> via <code>jco</code>)
T-07	time sink	<code>contracts.logs</code> , <code>maps.entrySet</code> , <code>maps.entrySet</code> , <code>maps.getStatus</code> , <code>maps.update</code> appear in no documentation page
T-08	time sink	Error-handling sample's catch-all arm collapses <code>PlaceholderDenied</code> and <code>PlaceholderNoUserContext</code> , whose distinction the WIT calls deliberate
T-09	time sink	Two doc URLs return 404; the payroll example page has no content
T-10	time sink	Reference repo version numbers disagree across <code>world.wit</code> , <code>Cargo.toml</code> , and a test name. The version lives in three places with no single source of truth — this bit me too
U-01	proposal	<code>set-claims-digest</code> deserves a documentation page of its own

18. One proposal

`kv-store.set-claims-digest` deserves a documentation page of its own.

Its WIT comment says the digest is planted in the transaction's Merkle leaf "so clients can verify receipts offline". That sentence answers the hardest question about confidential computing: how can anyone outside check work performed somewhere they cannot see? No ADK page discusses it.

This project is built on that capability, and the result holds up under an independent verifier — as section 12 shows, with the account at zero credits and the node no longer answering. Give it a page with a worked canonical-form

example, and the pitch for T3N changes from "trust the enclave" to "compute it yourself and compare". That is a materially stronger claim, and the machinery already ships.

19. Reproducing all of this

```
git clone https://github.com/bryankwandou/nullstamp
cd nullstamp/contract/z-tenant-nullstamp
cargo test --target "$($rustc -vV | sed -n 's/^host: //p')"
cargo build --target wasm32-wasip2 --release
wasm-tools component wit target/wasm32-wasip2/release/z_tenant_nullstamp.wasm

cd ../../scripts
npm install
npm run preflight                # no developer key needed

cp .env.example .env            # set T3N_API_KEY
npm run step:01                 # through step:11
npm run verify:offline -- ../submission/live-receipt.json
```

The offline verification at the end needs no key, no credits, and no network, so it can be run by anyone from a clean clone.

Reported by Bryan Kwandou — nayrbryangaming01@gmail.com